Aeroq CRISP-DM Research

Student Name: Joe O'Mahony
Student Number: 21788075
Module: Final Year Project Development 01
Course: Software Systems Development Year 4 Semester 1

Aviation Weather Centre (AWC)

[1]
```
import requests
import json
```

[2]
```
# AWC requests the use of a user agent, so I will provide
USER_AGENT = "Aeroq/0.1 (Educational use for final year project; 21788075@setu.ie)"

# AWC Navigational Data API Urls
awc_airport_url = "https://aviationweather.gov/api/data/airport"
```

[3]
```
# Airport ICAO ID constants
DUBLIN_AIRPORT_ICAO_CODE = "EIDW"
CORK_AIRPORT_ICAO_CODE = "EICK"
SHANNON_AIRPORT_ICAO_CODE = "EINN"

IRELAND_AIRPORTS_ICAO_CODES = [DUBLIN_AIRPORT_ICAO_CODE, CORK_AIRPORT_ICAO_CODE, SHANNON_AIRPORT_ICAO_CODE]
```

[4]
```
def get_awc_data(url, params, return_type="json", json_indent=2):
    try:
        response = requests.get(url, params=params)
        response.raise_for_status()

        if return_type.lower() == "json":
            data = response.json()
            return json.dumps(data, indent=json_indent)
        elif return_type.lower() == "json-obj":
            return response.json()
        elif return_type.lower() == "text":
            return response.text
        else:
            print("Invalid return type arg! Use JSON/text")
    except requests.exceptions.RequestException as re:
        print(f"Requests error: {re}")
    except json.JSONDecodeError as je:
        print(f"JSON decoding error: {je}")
    except Exception as err:
        print(f"An error occurred: {err}")
```

METARs

Meteorological Aerodrome Reports (METARs) are standardised weather reports providing *current* weather conditions most relevant to aviation for a given airport.

METARs are typically generated and published by Met Éireann every 30 minutes, at 20 and 50 minutes past.

Special weather reports (SPECI) may be issued where there is a notable swing in weather conditions.

[5]
```
awc_metar_url = "https://aviationweather.gov/api/data/metar"
```

Interpreting METARs

Full METAR interpretation guide (AWC): https://aviationweather.gov/help/data/#metar (redirect).

Raw Data:

`METAR EIDW 291600Z 15009KT 100V200 9999 -DZ SCT010 BKN013 OVC015 18/16 Q1001 NOSIG`

Interpretation:

Field	Value	Interpretation
Report Type	`METAR`	METAR
Place	`EIDW`	Dublin Airport
Date-Time Issued	`291600Z`	29th day, 16:00 UTC
Wind	`15009KT 100V200`	150° at 9 knots (variable 100°–200°)
Visibility	`9999`	10 km or greater
Phenomena	`-DZ`	Light drizzle
Clouds	`SCT010 BKN013 OVC015`	Scattered 1,000 ft, broken 1,300 ft, overcast 1,500 ft
Temperature	`18/16`	18°C with 16°C dewpoint
Pressure	`Q1001`	QNH 1001 hPa
Additional-Information	`NOSIG`	No significant changes expected in window

Dublin Airport (EIDW)

[6]
```
eidw_metar_response = get_awc_data(awc_metar_url, {"ids": DUBLIN_AIRPORT_ICAO_CODE, "format": "raw"}, return_type="text")
print("Dublin Airport (EIDW) Raw METAR Response:\n", eidw_metar_response)
```

```
Dublin Airport (EIDW) Raw METAR Response:
 METAR EIDW 281400Z 25008KT 200V280 9999 FEW021 SCT036 BKN042 14/10 Q1021 NOSIG
```

Cork Airport (EICK)

[7]
```
eick_metar_response = get_awc_data(awc_metar_url, {"ids": CORK_AIRPORT_ICAO_CODE, "format": "raw"}, return_type="text")
print("Cork Airport (EICK) Raw METAR Response:\n", eick_metar_response)
```

```
Cork Airport (EICK) Raw METAR Response:
 METAR EICK 281400Z 22011KT 9999 SCT012 BKN020 BKN180 14/12 Q1023 NOSIG
```

Shannon Airport (EINN)

[8]
```
einn_metar_response = get_awc_data(awc_metar_url, {"ids": SHANNON_AIRPORT_ICAO_CODE, "format": "raw"}, return_type="text")
print("Shannon Airport (EINN) Raw METAR Response:\n", einn_metar_response)
```

```
Shannon Airport (EINN) Raw METAR Response:
 METAR EINN 281400Z 23007KT 9999 FEW015 BKN050 15/11 Q1022 NOSIG
```

TAFs

Terminal Aerodrome Forecasts (TAFs) are standardised weather *forecasts* predicting the weather conditions most relevant to aviation for a given airport.

For our three Irish airports, TAFs are generated and published by Met Éireann at 05:00, 11:00, 17:00, and 23:00 UTC.

In most circumstances, a TAF for a given airport becomes active after 1-hour and is valid for 24-hours.

A TAF may be amended at any time due to changing weather conditions (`TAF AMD`), error corrections (`TAF COR`). Both TAF

amendments and TAF corrections invalidate and replace the preceding TAF. If TAF issuance is delayed, most often due to technical difficulties, it will be issued as a `TAF RTD`.

There are three change groups used in TAFs:

1. From (`FM`)

Indicates rapid, permanent changes **to** mean weather conditions taking place from a specified time. All elements of the forecast will be retransmitted.

2. Becoming (`BECMG`)

Indicates a gradual change in the weather over the specified period. Only the changing elements are listed.

3. Temporary (`TEMPO`)

Indicates a temporary change **from** mean weather conditions for less than half of the specified period. Additionally, `TEMPO` can only be used for periods of temporary fluctuation greater than 30 minutes, but less than 60. So, even if a `TEMPO` change is issued over a 4-hour period, the temporary fluctuation is only expected to last a maximum of 1-hour.

- Probability (`PROB`)

Used when there is a 30% probability (`PROB30`) **OR** 40% probability (`PROB40`) of a specified change in conditions over a time period. Probabilities less than 30% are not reported, probabilities over 40% should be issued as `TEMPO`, `FM`, or `BECMG`.

Sources:

https://www.tarmacview.com/glossary/taf/#taf-change-groups-fm-tempo-prob-becmg

https://skybrary.aero/sites/default/files/bookshelf/34407.pdf

[9] ✓ 0s

```
awc_taf_url = "https://aviationweather.gov/api/data/taf"
```

Interpreting TAFs

Full TAF interpretation guide (AWC): https://aviationweather.gov/help/data/#taf (redirect).

Raw Data:

```
TAF EICK 010500Z 0106/0206 23005KT 9999 FEW018
  PROB40
  TEMPO 0106/0109 -SHRA FEW018CB
  BECMG 0108/0110 20010KT
  TEMPO 0114/0202 20015G27KT
  TEMPO 0120/0202 4000 -RA BKN007 FEW018CB
  PROB40
  TEMPO 0121/0202 2500 RA BR BKN004
  PROB40
  TEMPO 0202/0206 2500 -SHRA BR BKN004
```

Base Forecast Interpretation:

Field	Value	Interpretation
Report Type	TAF	
Place	Cork Airport	
Date-Time Issued	1st of month, 05:00 UTC	
Validity Period	1st of the month 06:00 UTC until 2nd of the month 06:00 UTC	
Wind	230 degrees, 5 knots	
Visibility	9999 M or more	
Clouds	40% probability of a few clouds (~2/8ths cloud cover) at 1800 feet	

Change Groups Interpretation:

Change Type	Period	Value	Interpretation
Temporary	1st of the month, 06:00-09:00 UTC	`TEMPO 0106/0109 -SHRA FEW018CB`	Showers of rain; few cumulonimbus at 1800 ft.
Becoming	1st of the month, 08:00-10:00 UTC	`BECMG 0108/0110 20010KT`	Wind 200° at 10 knots

Dublin Airport (EIDW)

[10] ✓ 6s

```
eidw_taf_response = get_awc_data(awc_taf_url, {"ids": DUBLIN_AIRPORT_ICAO_CODE, "format": "raw"}, return_type="text")
print("Dublin Airport (EIDW) Raw TAF Response:\n", eidw_taf_response)
```

```
Dublin Airport (EIDW) Raw TAF Response:
 TAF EIDW 281100Z 2812/2912 23010KT 9999 SCT020 BKN030
  BECMG 2816/2818 19007KT
  TEMPO 2818/2822 16007KT
  BECMG 2900/2902 15005KT
  BECMG 2906/2908 16010KT
```

Cork Airport (EICK)

[11] ✓ 6s

```
eick_taf_response = get_awc_data(awc_taf_url, {"ids": CORK_AIRPORT_ICAO_CODE, "format": "raw"}, return_type="text")
print("Cork Airport (EICK) Raw TAF Response:\n", eick_taf_response)
```

```
Cork Airport (EICK) Raw TAF Response:
 TAF EICK 281100Z 2812/2912 23010KT 9999 SCT010 BKN040
  BECMG 2814/2816 20010KT
  PROB40
  TEMPO 2815/2819 4000 BR BKN008
  BECMG 2900/2902 17012KT
  TEMPO 2910/2912 17015G25KT
```

Shannon Airport (EINN)

[12] ✓ 3s

```
einn_taf_response = get_awc_data(awc_taf_url, {"ids": SHANNON_AIRPORT_ICAO_CODE, "format": "raw"}, return_type="text")
print("Shannon Airport (EINN) Raw TAF Response:\n", einn_taf_response)
```

```
Shannon Airport (EINN) Raw TAF Response:
 TAF EINN 281100Z 2812/2912 22010KT 9999 FEW012 BKN020
  BECMG 2815/2817 19007KT
  BECMG 2816/2818 16010KT
  TEMPO 2910/2912 17015G25KT
```

SIGMET

Significant Meteorological Information (SIGMET) is a non-routine severe weather advisory. SIGMETs use Flight Information Regions (FIRs) due to the nature of severe weather. A FIR is a large division of airspace in a region.

The Shannon FIR covers all of Ireland below Flight Level (FL) 245 (24,500 ft.). Above FL 245, Northern Ireland uses the Scottish FIR (EGPX).

[13] ✓ 0s

```
awc_isigmet_url = "https://aviationweather.gov/api/data/isigmet"
```

Fetch SIGMET by ICAO airport code prefix

SIGMET ICAO Code prefix to region table

Prefix	Region
A	Western South Pacific
B	Greenland, Iceland, and Kosovo (European Alternate)
C	Canada
D	Eastern parts of West Africa and Maghreb
E	Northern Europe
F	Most of Central Africa, Southern Africa, and the Indian Ocean

G	Western parts of West Africa and Maghreb
H	East Africa and Northeast Africa
K	Contiguous United States
L	Southern Europe, Israel, and Palestine
M	Central America, Mexico and northern/western parts of the Caribbean
N	Most of the South Pacific and New Zealand
O	Gulf States, Iran, Iraq, Pakistan, Jordan, West Bank
P	Most of the North Pacific, and Kiribati
R	Japan, South Korea, and the Philippines
S	South America
T	Eastern and southern parts of the Caribbean
U	Former USSR except the Baltics or Moldova
V	Many South Asian countries, mainland Southeast Asia, Hong Kong and Macau
W	Most of Maritime Southeast Asia
Y	Australia
Z	China, North Korea, and Mongolia

Adapted from the exhaustive list: https://en.wikipedia.org/wiki/ICAO_airport_code

Fetch Northern European ISIGMETs

[16] ✓ 0s

```
# I, J, Q, X never assigned
ICAO_CODE_PREFIX_VALID_REGION_SET = {"A", "B", "C", "D", "E", "F", "G", "H",
                                     "K", "L", "M", "N", "O", "P",
                                     "R", "S", "T", "U", "V", "W",
                                     "Y", "Z"}

def fetch_isigmet_region(region: str) -> list[object]:
    region = region.upper()
    if region not in ICAO_CODE_PREFIX_VALID_REGION_SET:
        Exception(f"Invalid region character! {region}")

    isigmet_response_arr = get_awc_data(awc_isigmet_url, {"format": "json"}, return_type="json-obj")
    filtered_isigmet_response_arr = []

    if isigmet_response_arr:
        for isigmet_obj in isigmet_response_arr:
            if isinstance(isigmet_obj, dict) and str(isigmet_obj.get("icaoId")).startswith(region):
                filtered_isigmet_response_arr.append(isigmet_obj)

    return filtered_isigmet_response_arr

northern_europe_isigmet_response_arr = fetch_isigmet_region('E')
print(json.dumps(northern_europe_isigmet_response_arr, indent=2) or "No current SIGMETs issued for E***")
```

```
[]
```

Fetch Irish ISIGMETs (Shannon FIR: EISN)

[17] ✓ 0s

```
SHANNON_FIR = "EISN"
```

[18] ✓ 0s

```
def fetch_isigmet_eisn(fir_id: str):
    region_isigmet_response_arr = fetch_isigmet_region(fir_id[0])
    filtered_isigmet_response_arr = []

    if region_isigmet_response_arr:
      for isigmet_obj in region_isigmet_response_arr:
        if isinstance(isigmet_obj, dict) and str(isigmet_obj.get("firId")).startswith(SHANNON_FIR):
          filtered_isigmet_response_arr.append(isigmet_obj)

    return filtered_isigmet_response_arr

eisn_isigmet_response = fetch_isigmet_eisn(SHANNON_FIR)
print(json.dumps(eisn_isigmet_response, indent=2) if eisn_isigmet_response else f"No current SIGMETs for Shannon airport FIR (EISN)")
```

```
No current SIGMETs for Shannon airport FIR (EISN)
```

Fetch Worldwide ISIGMETs (Excluding CONUS)

[23] ✓ 0s

```
def fetch_worldwide_isigmet():
    isigmet_response_arr = get_awc_data(awc_isigmet_url, {"format": "json"}, return_type="json-obj")
    filtered_isigmet_response_arr = []

    if isigmet_response_arr:
        for isigmet_obj in isigmet_response_arr:
            if isinstance(isigmet_obj, dict):
                filtered_isigmet_response_arr.append(isigmet_obj)

    return filtered_isigmet_response_arr

worldwide_isigmet = fetch_worldwide_isigmet()
print(f"Total worldwide SIGMETs (Excluding CONUS): {len(worldwide_isigmet)}")
print(json.dumps(worldwide_isigmet, indent=2))
```

```
Total worldwide SIGMETs (Excluding CONUS): 149
[
  {
    "icaoId": "MMMD",
    "firId": "MMFR",
    "firName": "MMFO MAZATLAN OCEANICA",
    "receiptTime": "2026-10-28T08:47:29.505Z",
    "validTimeFrom": 1793178000,
    "validTimeTo": 1793199600,
    "seriesId": "2",
    "hazard": "TC",
    "qualifier": "VANCE",
    "base": null,
    "top": 50000,
    "geom": "AREA",
    "coords": [
      {
        "lon": -109.779,
        "lat": 20.25
      },
      {
        "lon": -108.97,
        "lat": 19.38
      },
      {
        "lon": -107.57,
        "lat": 17.86
      },
      {
        "lon": -107.045,
        "lat": 17.256
      },
      {
        "lon": -106.983,
        "lat": 17.314
      },
      {
        "lon": -106.772,
        "lat": 17.6
      },
      {
        "lon": -106.617,
        "lat": 17.916
      },
      {
        "lon": -106.522,
        "lat": 18.253
      },
      {
```

```
          "lon": -106.49,
          "lat": 18.6
        },
        {
          "lon": -106.522,
          "lat": 18.947
        },
        {
```


OpenSky Network API

OpenSky blocks GCP servers, which Google Colab use.

To see the notebook researching the OpenSky API, please visit the `api/opensky` directory in the working branch of my fyp-code GitHub repo (https://github.com/JoeOMahony/fyp-code/tree/working).

AirLabs API

[19] ✓ 0s

```python
from google.colab import userdata
import urllib
import urllib.request
import json


def get_eidw_departures():
  api_key = f"api_key={userdata.get('AIRLABS_KEY')}"
  url = f"https://airlabs.co/api/v9/flights?dep_icao={DUBLIN_AIRPORT_ICAO_CODE}&{api_key}"

  req = urllib.request.Request(url, headers={'User-Agent': USER_AGENT})
  try:
      with urllib.request.urlopen(req, timeout=10) as response:
        dub = json.load(response)
        print(json.dumps(dub, indent=4))
  except Exception as e:
    print(f"Error: {e}")

get_eidw_departures()
```


```
{
    "request": {
        "lang": "en",
        "currency": "USD",
        "time": 1,
        "id": "pvvprkr00w0",
        "server": "a",
        "host": "airlabs.co",
        "pid": 1182822,
        "key": {
            "id": 49966,
            "api_key": "a50c8a0e-47db-4d50-8035-aeab77799371",
            "type": "free",
            "expired": "2026-11-12T00:00:00.000Z",
            "registered": "2026-10-12T13:10:17.000Z",
            "upgraded": null,
            "limits_by_hour": 2500,
            "limits_by_minute": 250,
            "limits_by_month": 1000,
            "limits_total": 988
        },
        "params": {
            "dep_icao": "EIDW",
            "lang": "en"
        },
        "version": 9,
        "method": "flights",
        "client": {
            "ip": "136.70.42.120",
            "geo": {
                "country_code": "US",
                "country": "United States",
                "continent": "North America",
                "lat": 37.751,
                "lng": -97.822,
                "timezone": "America/Chicago"
            },
            "connection": {
                "type": "corporate"
            },
            "device": {},
            "agent": {},
            "karma": {
                "is_blocked": false,
                "is_crawler": false,
                "is_bot": false,
                "is_friend": false,
                "is_regular": true
            }
        }
    },
    "response": [
        {
            "hex": "400960",
            "reg_number": "G-TTOE",
            "flag": "GB",
            "lat": 51.752534,
            "lng": -0.634587,
```


Colab paid products - Cancel Google AI contracts here

Variables Terminal ✓ 14:49 Python 3